3. Water Jug Problem (BFS Approach)

```
from collections import deque

def solve_water_jug(j1_cap, j2_cap, target):
    queue = deque([((0, 0), [])])
    visited = set()

    while queue:
        (curr_j1, curr_j2), path = queue.popleft()
        if (curr_j1, curr_j2) in visited: continue
        visited.add((curr_j1, curr_j2))
        current_path = path + [(curr_j1, curr_j2)]

        if curr_j1 == target or curr_j2 == target: return current_path

        moves = [
            (j1_cap, curr_j2), (curr_j1, j2_cap), (0, curr_j2), (curr_j1, 0),
            (curr_j1 - min(curr_j1, j2_cap - curr_j2), curr_j2 + min(curr_j1, j2_cap - curr_j2)),
            (curr_j1 + min(curr_j2, j1_cap - curr_j1), curr_j2 - min(curr_j2, j1_cap - curr_j1))
        ]
        for move in moves:
            if move not in visited: queue.append((move, current_path))
    return None

if __name__ == "__main__":
    j1, j2, target = 4, 3, 2
    solution = solve_water_jug(j1, j2, target)
    if solution:
        for step, state in enumerate(solution):
            print(f"Step {step}: Jug1 = {state[0]}L, Jug2 = {state[1]}L")
```

main.py

Visualize | Run

```python
def solve_water_jug(jug1_cap, jug2_cap, target):
    queue = deque([(0, 0, [(0, 0)])])
    visited = {(0, 0)}

    while queue:
        j1, j2, path = queue.popleft()

        if j1 == target or j2 == target:
            return path
        moves = [
            (jug1_cap, j2),
            (j1, jug2_cap),
            (0, j2),
            (j1, 0),
            (j1 - min(j1, jug2_cap - j2), j2 + min(j1, jug2_cap - j2)),
            (j1 + min(j2, jug1_cap - j1), j2 - min(j2, jug1_cap - j1))
        ]

        for next_j1, next_j2 in moves:
            if (next_j1, next_j2) not in visited:
                visited.add((next_j1, next_j2))
```

Output

```
Steps to get exactly 2L using 4L and 3L jugs:
Step 0: Jug1 = 0L, Jug2 = 0L
Step 1: Jug1 = 0L, Jug2 = 3L
Step 2: Jug1 = 3L, Jug2 = 0L
Step 3: Jug1 = 3L, Jug2 = 3L
Step 4: Jug1 = 4L, Jug2 = 2L
```